Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación a Distancia

Programación II

TRABAJO PRÁCTICO INTEGRADOR

Food Store

Sistema de gestión de pedidos de comida

Integrantes

Nombre y Apellido	Legajo	DNI
Felipe Bessega	54197	48812874
Alvaro David Salla	54260	47968070
Santiago Jalil	54240	48542162

Docentes	Cinthia Rigoni
	Ariel Enferrel
Fecha de entrega	9/11/2026

Índice

1. Introducción 3
2. Marco teórico 4
    2.1 Python y type hints 4
    2.2 Clases y objetos 4
    2.3 Encapsulamiento 4
    2.4 Herencia y clases abstractas 4
    2.5 Interfaces 4
    2.6 Polimorfismo y sobrescritura 4
    2.7 Asociación, agregación y composición 5
    2.8 Excepciones 5
    2.9 Colecciones y enumeraciones 5
3. Decisiones técnicas y arquitectura 6
    3.1 Cómo se abordó la solución 6
    3.2 Organización del proyecto 6
    3.3 Modelo de datos 6
    3.4 Servicio y reglas de negocio 7
    3.5 Vistas, herencia y polimorfismo 8
    3.6 Manejo de errores 8
    3.7 Versión web 9
    3.8 Capturas del sistema funcionando 9
4. Dificultades y soluciones 15
5. Bibliografía y webgrafía 16
6. Enlaces 17

1. Introducción

Food Store es un sistema para gestionar un negocio de comidas: categorías, productos, usuarios y pedidos. Se desarrolló en Python aplicando el paradigma de Programación Orientada a Objetos (POO), como Trabajo Práctico Integrador de Programación II.

El sistema arranca con un inicio de sesión. Según el rol del usuario se abre la sección de administrador, que permite dar de alta, modificar, dar de baja y consultar todas las entidades, o la sección de cliente, que solo permite consultar. Los datos viven en memoria mientras el programa está abierto; al cerrarlo vuelven a los valores de ejemplo.

A diferencia de la consigna original, que propone un menú de consola, la interfaz se construyó como una aplicación gráfica de escritorio con la biblioteca Flet. La sección 3 explica esta decisión.

2. Marco teórico

2.1 Python y type hints

Python es un lenguaje interpretado, de tipado dinámico y multiparadigma, que soporta POO de forma nativa. Su biblioteca estándar incluye módulos usados en este trabajo, como `abc` (clases abstractas), `enum` (enumeraciones), `dataclasses` y `datetime`.

Los type hints (PEP 484) son anotaciones que indican el tipo esperado de atributos, parámetros y valores de retorno, por ejemplo `def autenticar(self, mail: str, clave: str) -> Usuario | None`. Python no los verifica al ejecutar, pero documentan el contrato de cada función y permiten que el editor y herramientas como mypy detecten errores antes de correr el programa.

2.2 Clases y objetos

Una clase define la estructura (atributos) y el comportamiento (métodos) de un tipo de objeto. Un objeto es una instancia concreta de esa clase. El constructor `__init__` inicializa el estado del objeto al crearlo.

2.3 Encapsulamiento

El encapsulamiento consiste en ocultar el estado interno de un objeto y exponerlo solo a través de una interfaz controlada. En Python, un atributo con doble guion bajo (`__clave`) queda privado por *name mangling*, y el decorador `@property` permite exponer un valor como si fuera un atributo, con un setter opcional que valida el dato antes de guardarlo.

2.4 Herencia y clases abstractas

La herencia permite que una clase (subclase) reutilice y extienda los atributos y métodos de otra (superclase). Una clase abstracta, que en Python hereda de `ABC`, no se puede instanciar: sirve como molde común y obliga a sus subclases a implementar los métodos marcados con `@abstractmethod`.

2.5 Interfaces

Una interfaz define un conjunto de métodos que una clase se compromete a implementar, sin decir cómo. Python no tiene una palabra reservada para interfaces; se modelan como clases abstractas sin atributos y con todos sus métodos abstractos. La consigna propone la interfaz `Calculable`, con el método `calcular_total()`.

2.6 Polimorfismo y sobrescritura

El polimorfismo permite tratar objetos de clases distintas a través de una misma interfaz: el código llama a un método sin saber qué clase concreta lo resuelve. La sobrescritura es el mecanismo que lo hace posible: una subclase redefine un método heredado (por ejemplo `__str__`, `__eq__` o un método abstracto) con su propio comportamiento.

2.7 Asociación, agregación y composición

Son relaciones entre clases. En la asociación un objeto conoce a otro (un pedido conoce a su usuario). La agregación es una relación todo-parte en la que las partes pueden existir sin el todo (una categoría agrupa productos, pero el producto existe por sí mismo). La composición es una relación todo-parte fuerte: la parte nace y muere con el todo (un detalle de pedido no tiene sentido fuera de su pedido).

2.8 Excepciones

Una excepción es un objeto que representa un error ocurrido durante la ejecución. Se lanza con `raise` y se captura con `try/except`, lo que permite informar el problema y continuar en lugar de cerrar el programa. Las excepciones propias se crean heredando de `Exception` y permiten distinguir los errores de negocio de los errores de programación.

2.9 Colecciones y enumeraciones

Las colecciones agrupan objetos en memoria. Python ofrece listas (secuencias ordenadas), tuplas (secuencias inmutables), conjuntos (elementos únicos) y diccionarios (pares clave-valor, con búsqueda por clave en tiempo constante). Las enumeraciones (`enum.Enum`) definen un conjunto cerrado de valores con nombre, como los roles o los estados de un pedido, y evitan usar textos sueltos que pueden escribirse mal.

3. Decisiones técnicas y arquitectura

3.1 Cómo se abordó la solución

El trabajo se empezó por el diseño de las pantallas y después se construyó la lógica que las sostiene. Se eligió una interfaz gráfica con Flet en lugar de la consola porque permite mostrar los errores de validación junto al campo que los causó, usar listas desplegables para los enumerados y tablas para los listados.

La regla principal de la arquitectura es que la interfaz no valida ni guarda datos: cada pantalla llama a un método del servicio y, si algo es inválido, el servicio lanza una excepción propia que la pantalla muestra.

3.2 Organización del proyecto

```
main.py                 Punto de entrada: abre la ventana y crea la App
pyproject.toml          Datos del proyecto y dependencias
assets/fonts/           Tipografía Inter
web/                    Versión web compilada (la publica Vercel)
UI/
├── app.py              Navegación entre pantallas y sesión del usuario
├── tema.py             Colores, tipografía y formato de precios
├── datos_demo.py       Enumerados, excepciones, modelos, servicio y datos de ejemplo
├── componentes/        Piezas reutilizables: botón, campo, tabla, alerta, modal...
└── vistas/             Una clase por pantalla
    ├── vista_abc.py    Clases base abstractas y los dos layouts (admin y cliente)
    ├── login.py
    ├── admin_inicio.py, admin_categorias.py, admin_productos.py,
    │   admin_usuarios.py, admin_pedidos.py
    └── cliente.py
```

La dependencia va en un solo sentido: las vistas usan el servicio y el servicio usa los modelos. Ningún modelo ni el servicio conocen a Flet.

3.3 Modelo de datos

Los modelos, los enumerados y las excepciones están en `UI/datos_demo.py`.

Clase	Atributos	Detalle
Categoria	id, nombre, descripcion, eliminado	`@dataclass`.
Producto	id, nombre, descripcion, precio, stock, categoria_id, imagen, disponible, eliminado	`@dataclass`. Se relaciona con su categoría por id (N..1).
Usuario	id, nombre, apellido, mail, celular, __clave, rol, eliminado	La clave es privada y no tiene getter; se compara con `verificar_clave()` y se cambia con `cambiar_clave()`. Properties `nombre_completo` e `iniciales`.

Clase	Atributos	Detalle
Pedido	id, usuario_id, estado, forma_pago, fecha, detalles, eliminado	`total` es una property que suma los subtotales; `cerrado` indica si está TERMINADO o CANCELADO.
DetallePedido	producto_id, nombre, cantidad, precio	Guarda el nombre y el precio del producto al momento de agregarlo; `subtotal` = cantidad × precio.

Enumerados: `Rol` (ADMIN, USUARIO), `Estado` (PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO) y `FormaPago` (TARJETA, TRANSFERENCIA, EFECTIVO). Las transiciones de estado permitidas se definen en un único diccionario, `TRANSICIONES_VALIDAS`, que relaciona cada estado con el conjunto de estados a los que puede pasar.

Que el detalle copie el precio del producto cumple dos objetivos: el subtotal no depende de un dato que escriba el operador, y un pedido viejo sigue mostrando el precio que se cobró aunque el producto cambie de precio o se dé de baja.

3.4 Servicio y reglas de negocio

La clase `ServicioDemo` guarda cuatro diccionarios privados (`_categorias`, `_productos`, `_usuarios`, `_pedidos`) indexados por id, y concentra todas las reglas de negocio. Se usaron diccionarios porque la mayoría de las operaciones buscan por id. El id nuevo se calcula como el mayor existente más uno, con un contador independiente por colección.

Regla	Dónde se aplica
Precio y stock no pueden ser negativos	`_validar_producto()`
Nombre de categoría único sin distinguir mayúsculas	`_validar_categoria()`
No se elimina una categoría con productos activos (se informa cuántos tiene)	`eliminar_categoria()`
Mail con formato válido y único; contraseña de 6 caracteres o más	`_validar_usuario()`
Un administrador no puede eliminarse a sí mismo	`eliminar_usuario()`
Cantidad mayor a 0 y no mayor al stock, sumando lo que ya está en el pedido	`validar_detalle()`
Crear un pedido es todo o nada: si un detalle falla, el stock no cambia	`crear_pedido()`
Solo las transiciones de `TRANSICIONES_VALIDAS`	`cambiar_estado()`
Cancelar, o eliminar un pedido no cerrado, devuelve el stock	`cambiar_estado()`, `eliminar_pedido()`
Toda baja es lógica (`eliminado = True`)	Todos los métodos `eliminar_*`

Regla	Dónde se aplica
Un usuario eliminado no puede iniciar sesión	`autenticar()` recorre solo usuarios activos

`crear_pedido()` valida todos los detalles antes de descontar una sola unidad de stock. Recién cuando todos son válidos descuenta el stock y agrega el pedido a la colección, así un error a mitad de camino no deja el sistema en un estado inconsistente.

3.5 Vistas, herencia y polimorfismo

Cada pantalla es una clase. La jerarquía es la siguiente:

```
Vista (ABC)                    construir() abstracto
└── VistaConMenu (ABC)         contenido() y layout() abstractos; refrescar()
    ├── VistaAdmin             layout con barra lateral
    │   └── VistaAdminCategorias, VistaAdminProductos, VistaAdminUsuarios,
    │       VistaAdminPedidos, VistaAdminInicio y los formularios
    └── VistaCliente           layout con barra superior
        └── VistaClienteCategorias, VistaClienteProductos,
            VistaClientePedidos, VistaClienteDatos
VistaLogin(Vista)
```

`VistaConMenu.construir()` arma la pantalla llamando a `contenido()` y `layout()` sin saber qué subclase los resuelve: cada pantalla solo define su contenido y hereda el menú de su rol. Así, el menú de administrador y el de cliente son dos clases distintas con la misma interfaz.

La clase `App` registra las pantallas en el diccionario `RUTAS` y las abre con `ir(ruta)`, que además redirige al inicio de sesión si no hay usuario. El rol se consulta en un único lugar, `App.iniciar_sesion()`, para decidir la primera pantalla. El cliente no tiene ninguna pantalla que modifique datos: su menú solo enlaza vistas de consulta.

La sesión no es una variable global: el usuario logueado se guarda en el atributo `usuario` de la instancia de `App`, que se pasa a cada vista. Cerrar sesión lo pone en `None` y vuelve al login sin recrear el servicio, por lo que los datos cargados se conservan.

3.6 Manejo de errores

Hay dos excepciones propias:

- `FoodStoreError`: error de negocio. Además del mensaje guarda opcionalmente el `campo` que lo causó, para marcar ese input en rojo, y un `titulo` para la alerta.
- `ErroresValidacion` (hereda de `FoodStoreError`): agrupa todos los errores de un formulario en un diccionario `{campo: mensaje}`, para mostrarlos todos a la vez en lugar de uno por intento.

Las conversiones de texto a número se hacen en `_a_numero()`, que captura `ValueError` y lo convierte en un `FoodStoreError` con un mensaje claro. Las vistas capturan `FoodStoreError` y muestran el resultado de cada operación en una alerta de éxito o de error; ningún dato mal ingresado cierra el programa.

El inicio de sesión permite tres intentos y muestra el mismo mensaje sin importar qué dato falló. Al tercer error informa que se agotaron los intentos y cierra la aplicación.

3.7 Versión web

Flet permite compilar la misma aplicación para el navegador: Python corre dentro de la página con Pyodide. La versión compilada se guarda en la carpeta `web/` y Vercel la publica tal cual, sin compilar nada, en https://tpifacu.vercel.app. Cada visitante tiene su propia copia de los datos de ejemplo.

3.8 Capturas del sistema funcionando



Figura 1. Inicio de sesión.



Figura 2. Credenciales incorrectas: mensaje genérico y barra de intentos.

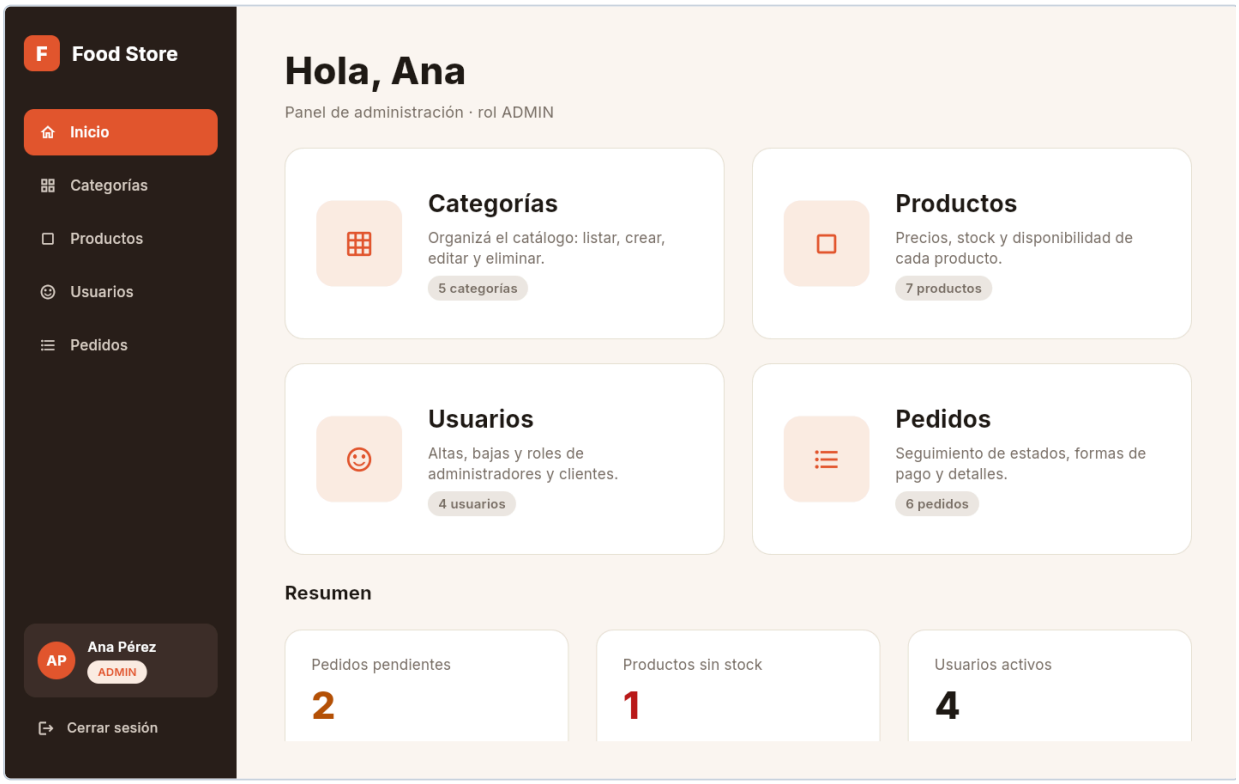


Figura 3. Inicio del administrador.

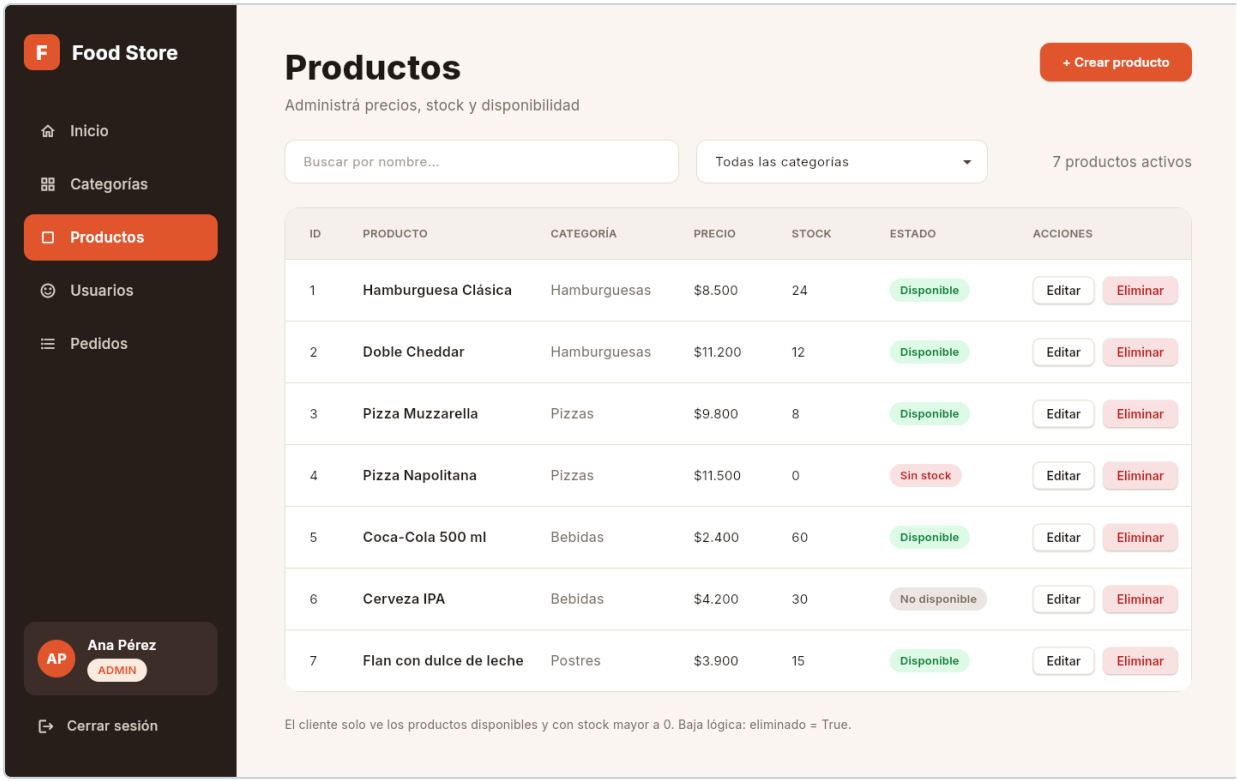


Figura 4. Listado de productos del administrador.

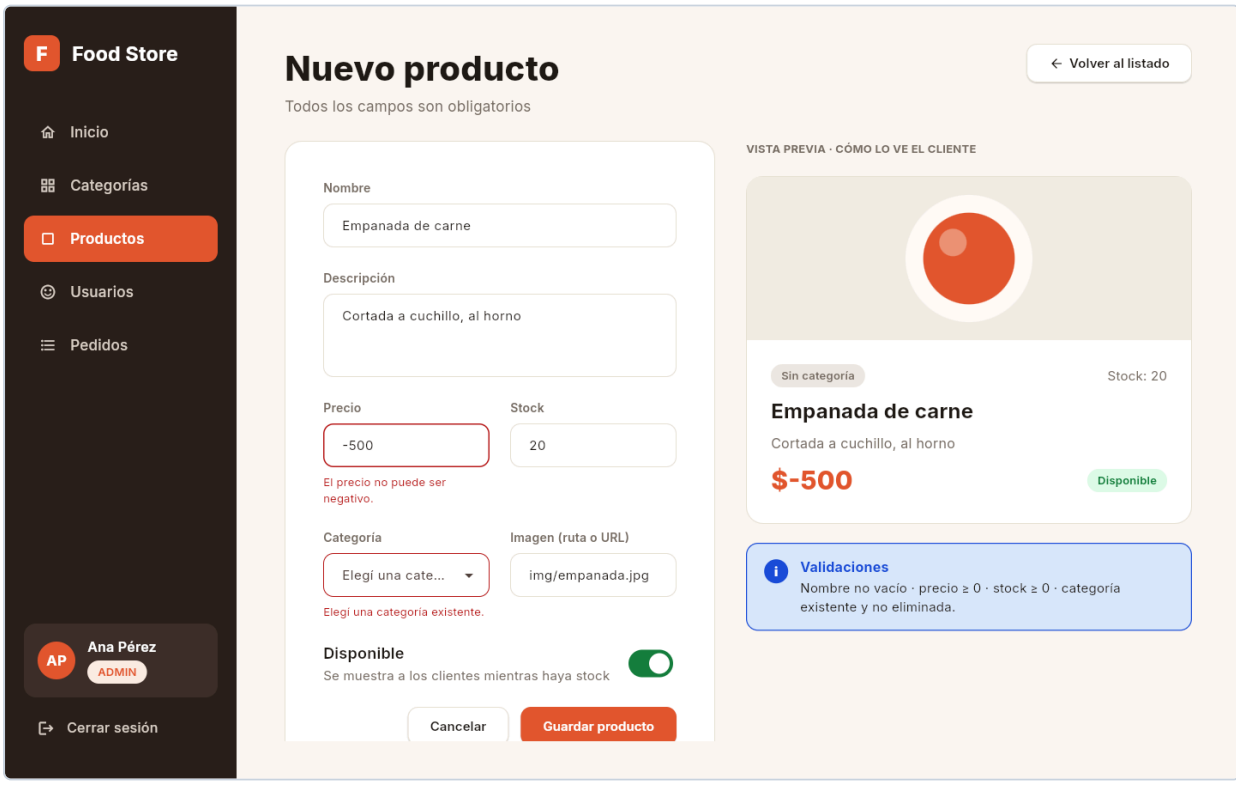


Figura 5. Alta de producto con errores de validación: precio negativo y categoría sin elegir.

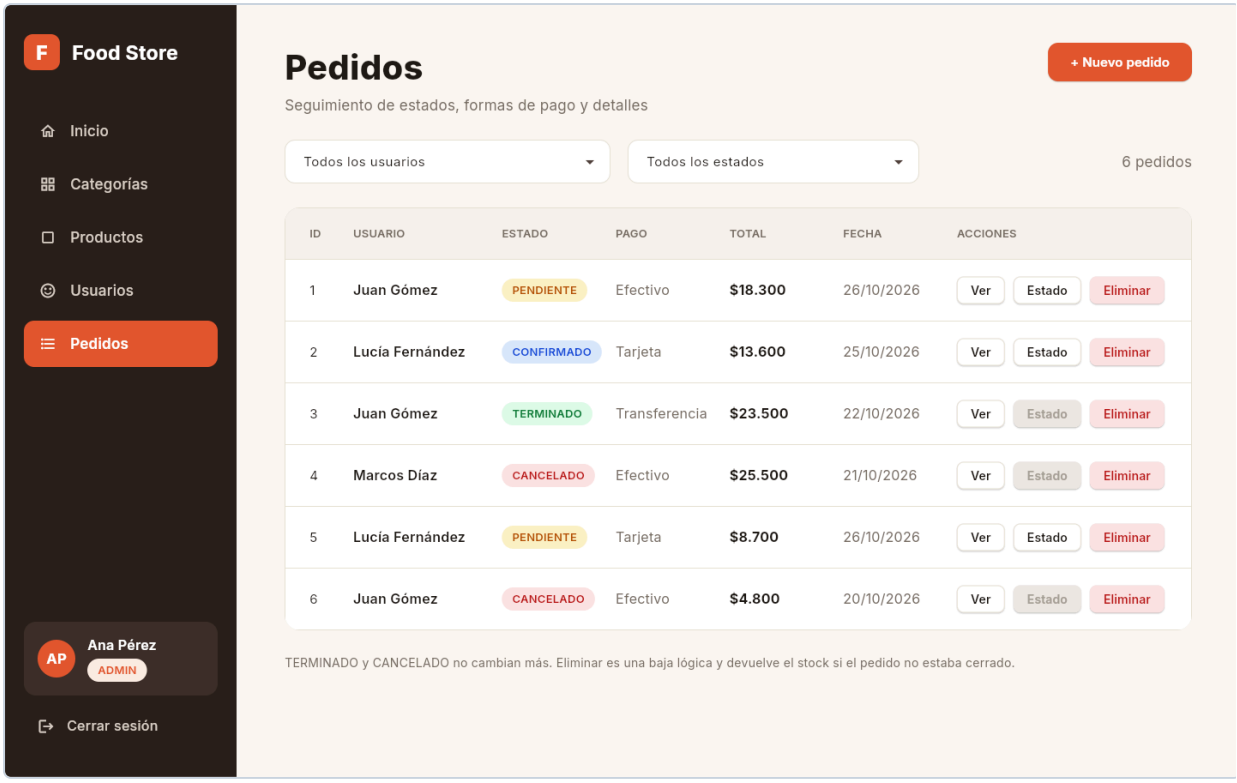


Figura 6. Listado de pedidos con sus estados.



Figura 7. Cambio de estado rechazado: PENDIENTE → TERMINADO no es una transición válida.

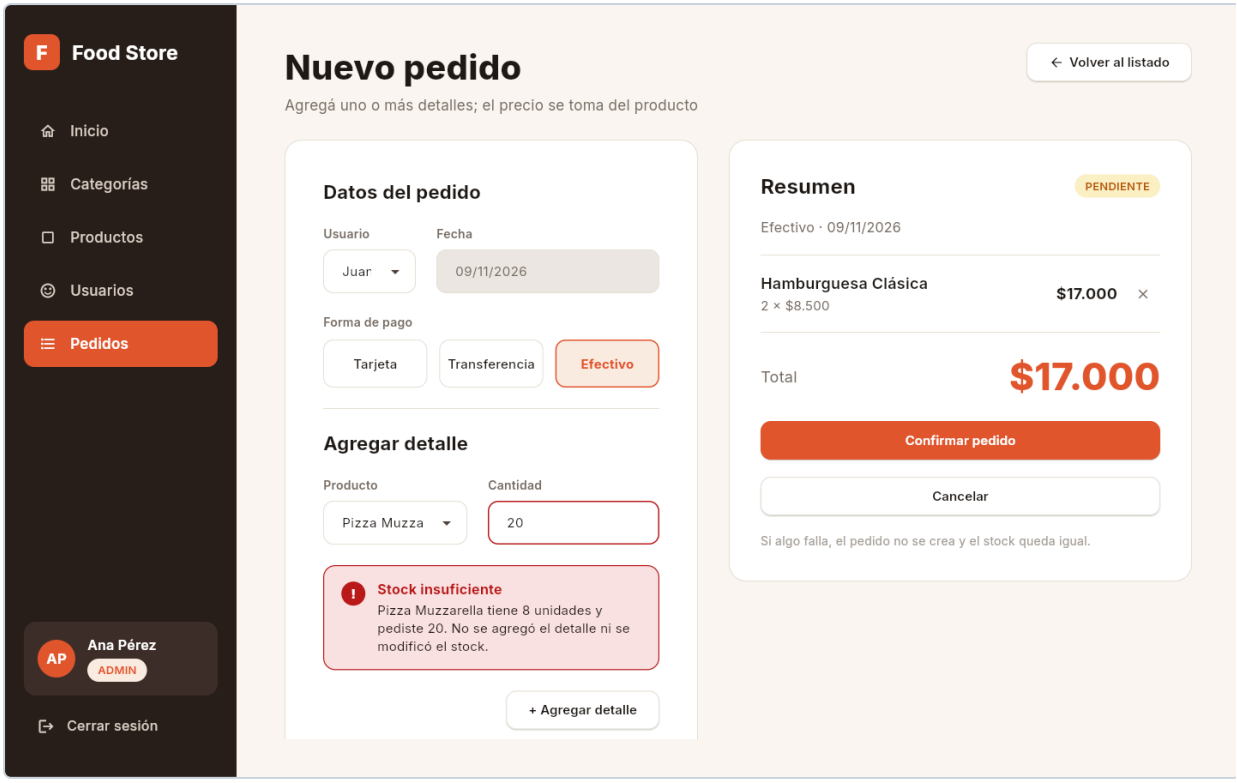


Figura 8. Alta de pedido: el detalle se rechaza por stock insuficiente y el stock no cambia.

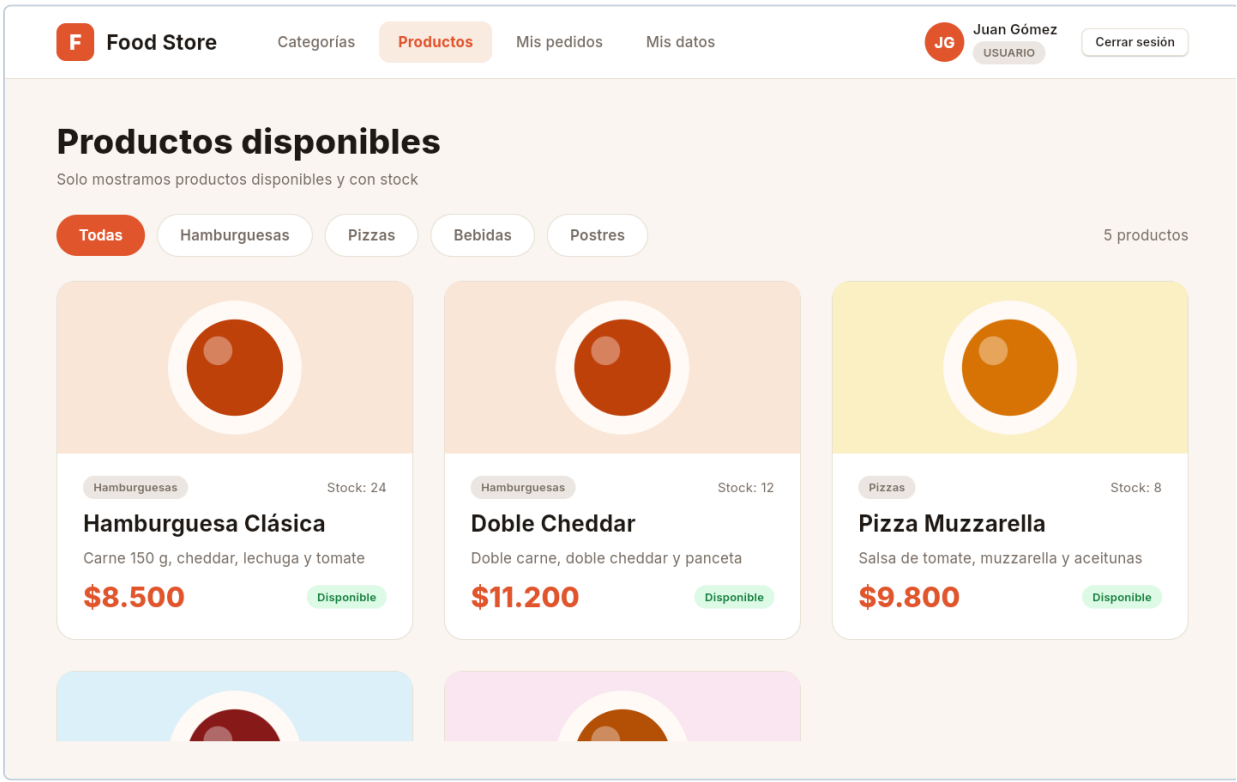


Figura 9. Vista del cliente: solo productos disponibles y con stock.

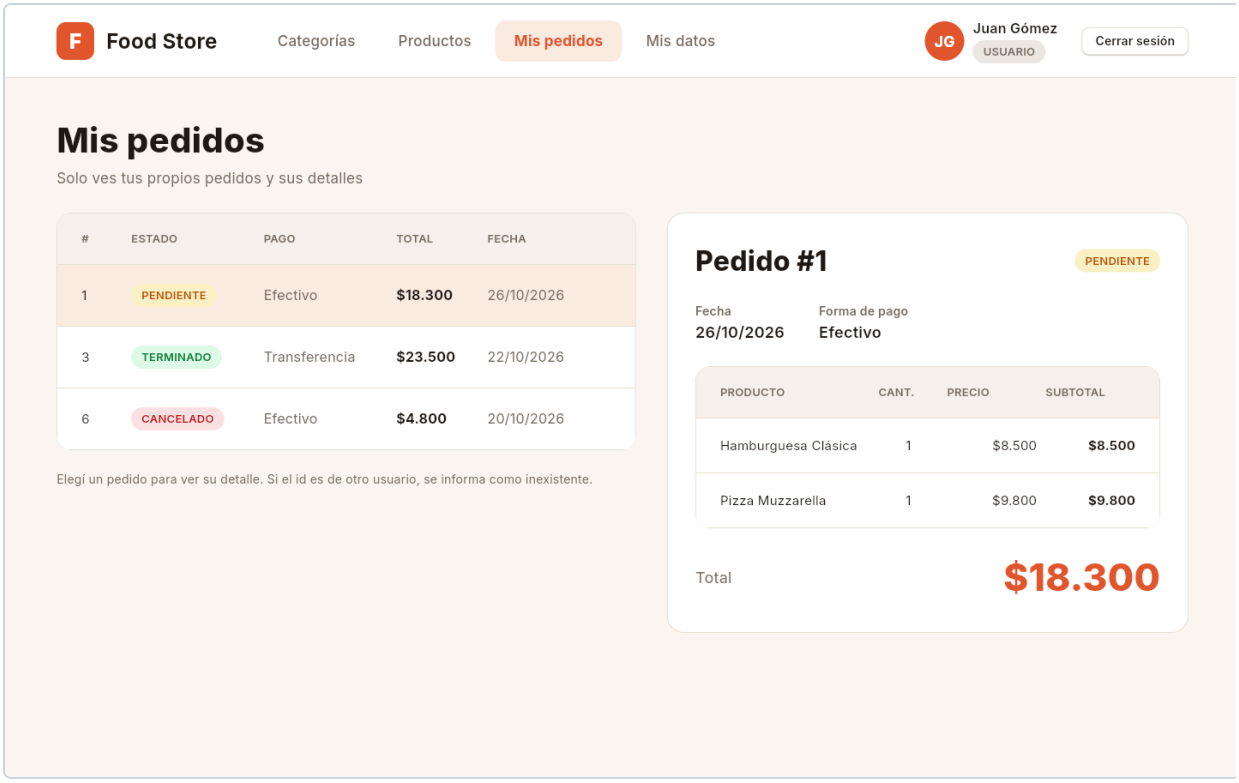


Figura 10. Vista del cliente: sus pedidos y el detalle del seleccionado.

4. Dificultades y soluciones

Dificultad	Solución
Mostrar varios errores de un formulario a la vez, cada uno junto a su campo.	Se creó `ErroresValidacion`, que junta todos los errores en un diccionario por campo; la vista recorre ese diccionario y marca cada input.
Evitar que un pedido con un detalle inválido descuente parte del stock.	`crear_pedido()` valida todos los detalles antes de modificar nada (todo o nada).
Al agregar dos veces el mismo producto al pedido se podía superar el stock.	`validar_detalle()` recibe la cantidad que ya está en el pedido y controla la suma contra el stock.
Ejecutar `UI/app.py` directamente desde VS Code fallaba porque Python no encontraba el paquete `UI`.	`app.py` agrega la carpeta raíz del proyecto a `sys.path` antes de importar.
En el navegador una página no puede cerrarse sola al agotar los intentos de login.	En la versión web se muestra que el programa se cerró y se indica recargar la página.
Publicar la versión web en Vercel.	Se compila localmente con `flet build web`, se copia el resultado a `web/` y Vercel solo sirve esos archivos (`vercel.json`).

5. Bibliografía y webgrafía

- Python Software Foundation. *Documentación oficial de Python 3*. https://docs.python.org/3/
- Python Software Foundation. *abc: Abstract Base Classes*. https://docs.python.org/3/library/abc.html
- Python Software Foundation. *enum: Support for enumerations*. https://docs.python.org/3/library/enum.html
- Python Software Foundation. *dataclasses: Data Classes*. https://docs.python.org/3/library/dataclasses.html
- van Rossum, G., Lehtosalo, J. y Langa, Ł. *PEP 484: Type Hints*. https://peps.python.org/pep-0484/
- Flet. *Documentación oficial*. https://flet.dev/docs/
- Vercel. *Documentación oficial*. https://vercel.com/docs
- Universidad Tecnológica Nacional. *Trabajo Práctico Integrador: Food Store*. Programación II.

6. Enlaces

Recurso	Enlace
Repositorio	https://github.com/felipipee/TPI-Facultad-C2
Versión web	https://tpifacu.vercel.app

Credenciales del administrador inicial: `admin@foodstore.com` / `admin123`. Clientes de prueba: `juan.gomez@mail.com` / `juan123`, `lucia.f@mail.com` / `lucia123`, `marcos.diaz@mail.com` / `marcos123`.